



8.3. Special Operations on Linked Lists

Previous Sections:

 [8. Linked Lists](#)

 [8.1. Singly Linked Lists](#)

 [8.2. Doubly Linked Lists](#)

Contents:

[1. Reversing a Linked List](#)

[2. Concatenating Linked Lists](#)

[3. Detecting Cycles in a Linked List](#)

[3.1. Cycle Detection Using a Set](#)

[3.2. Floyd's Cycle Detection Algorithm](#)

[Summary](#)

[References](#)

In the previous sections, we learned how to perform the fundamental operations on Singly Linked Lists and Doubly Linked Lists, including traversal, searching, insertion, and deletion. These operations form the foundation of linked list manipulation and are sufficient for many common applications.

However, real-world applications often require more advanced operations. For example, we may need to reverse the order of nodes, combine two linked lists into one larger list, or determine whether a linked list contains a cycle. These operations appear frequently in software engineering interviews, algorithm design, operating systems, and memory management.

In this section, we will explore several useful linked list operations:

- Reversing a linked list
- Concatenating two linked lists
- Detecting cycles in a linked list
- Floyd's Cycle Detection Algorithm

The examples in this section use a Doubly Linked List implementation because it provides both `next` and `prev` references, making some operations easier to visualize.

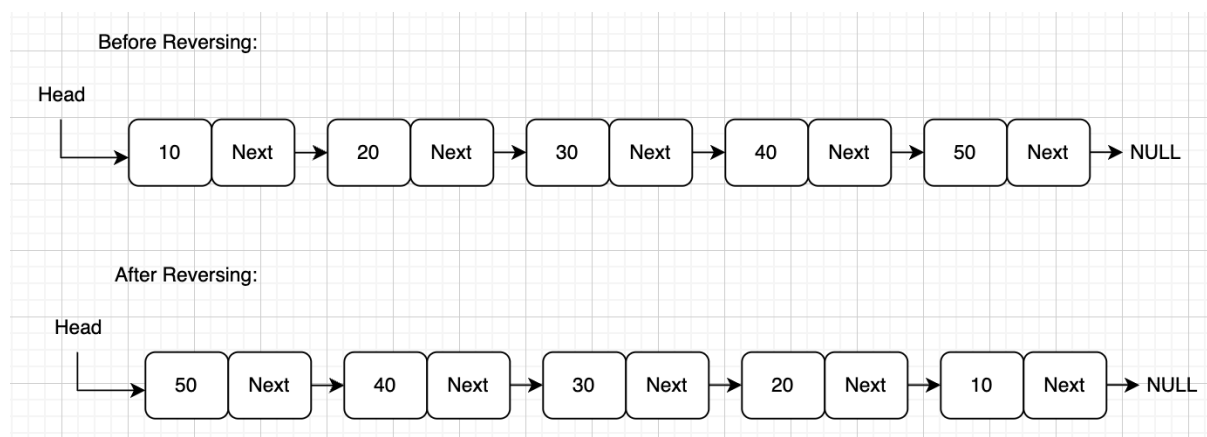
```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None

class LinkedList:
    def __init__(self):
        self.head = None
```

1. Reversing a Linked List

Reversing a linked list means changing the direction of every connection between nodes so that the first node becomes the last node and the last node becomes the first node.

For example:



Instead of creating a new linked list, we can reverse the existing list by modifying the links between nodes.

The algorithm uses three logical references:

- **current** — the node currently being processed.
- **previous** — the node behind the current node.
- **next** — the node ahead of the current node.

For a Doubly Linked List, the process becomes even simpler because each node already contains both `next` and `prev` references. We simply swap these references for every node.

Algorithm:

1. Start at the head node.
2. Swap the `next` and `prev` references.
3. Move to the next node in the original sequence.
4. Continue until all nodes have been processed.
5. Update the head pointer.

Time Complexity: $O(n)$ because every node must be visited exactly once.

Implementation:

```
def reverse(self):
    current = self.head
    temp = None

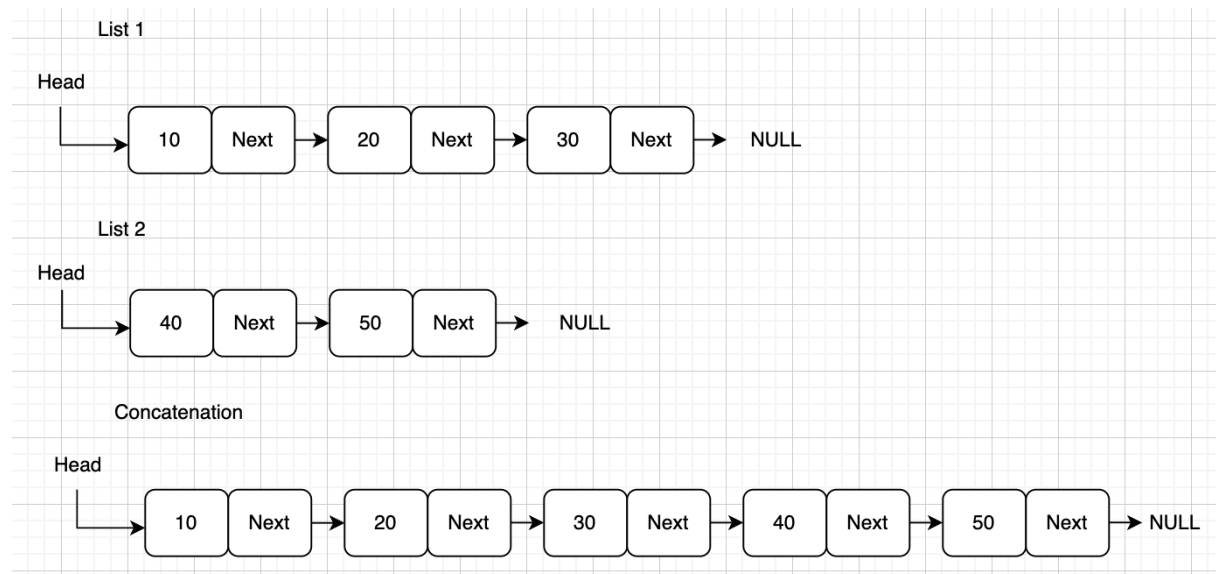
    while current is not None:
        temp = current.prev
        current.prev = current.next
        current.next = temp

        current = current.prev

    if temp is not None:
        self.head = temp.prev
```

2. Concatenating Linked Lists

Concatenation means joining two linked lists together to form a single larger list. Suppose we have two linked lists:



The basic idea is simple:

1. Locate the last node of the first list.
2. Connect it to the head of the second list.
3. Return the resulting list.

Time Complexity: $O(n)$ where n is the length of the first linked list.

Implementation:

```
def copy(self):  
  
    new_list = LinkedList()  
    if self.head is None:  
        return new_list  
  
    # Copy the head node  
    current = self.head  
    new_node = Node(current.data)  
    new_list.head = new_node  
    new_current = new_node
```

```

# Traverse and copy remaining nodes
current = current.next
while current:
    next_node = Node(current.data)
    new_current.next = next_node
    next_node.prev = new_current
    new_current = next_node
    current = current.next

return new_list

```

```

def concatenate(self, another):
    list1 = self.copy()
    list2 = another.copy()

    current = list1.head

    while current.next:
        current = current.next

    current.next = list2.head

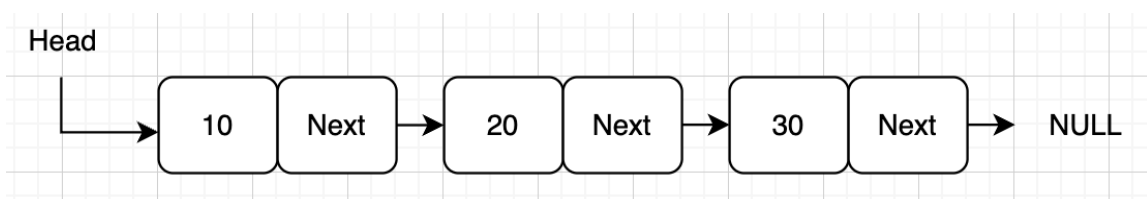
    return list1

```

3. Detecting Cycles in a Linked List

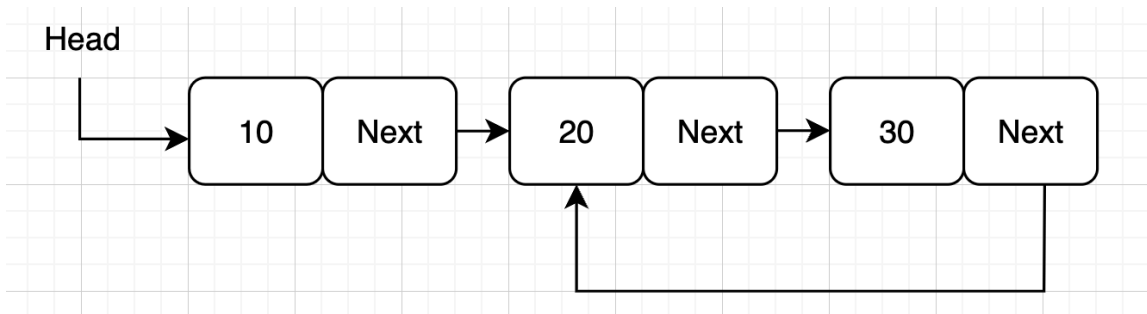
A linked list normally ends when the final node points to `None`.

Example of a normal linked list:



However, mistakes in programming or special applications may create a cycle.

Example of a cyclic linked list:



In this case: The list never reaches `None`. Traversal continues forever. Standard traversal algorithms can become stuck in an infinite loop.

Therefore, it is important to determine whether a linked list contains a cycle.

3.1. Cycle Detection Using a Set

One simple approach is to remember every node that has already been visited.

The algorithm works as follows:

1. Create an empty set.
2. Traverse the linked list.
3. Add every visited node into the set.
4. If a node is encountered again, a cycle exists.
5. If `None` is reached, no cycle exists.

Time Complexity: $O(n)$; Space Complexity: $O(n)$ because a set is used to store visited nodes.

Implementation:

```
def check_cycle(self):
    nodes = set()
    current = self.head

    while True:
        if (current is None) or (current in nodes):
            break

        nodes.add(current)
        current = current.next
```

```
if current is None:
    return False
else:
    return True
```

3.2. Floyd's Cycle Detection Algorithm

Although the set-based solution is simple, it requires additional memory. A more elegant solution is **Floyd's Cycle Detection Algorithm**, also known as the **Tortoise and Hare Algorithm**.

The algorithm uses two pointers: **Slow Pointer** (Tortoise); **Fast Pointer** (Hare). The pointers start at the head of the list. During traversal: Slow moves one node at a time.; Fast moves two nodes at a time.

- **Case 1 - No Cycle:** The fast pointer eventually reaches `None`.
- **Case 2 - Cycle Exists:** Since the fast pointer moves faster, it eventually catches the slow pointer inside the cycle.

Imagine two runners on a circular track: One runner moves slowly. The other runner moves twice as fast. Eventually, the faster runner must lap the slower runner. The same idea applies to nodes in a cyclic linked list.

Time Complexity: $O(n)$; Space Complexity: $O(1)$

Implementation:

```
def floyd_cycle_algorithm(head):

    slow = head
    fast = head

    while fast and fast.next:

        slow = slow.next
        fast = fast.next.next
```

```
        if slow == fast:
            return True

    return False
```

Unlike the set-based solution, Floyd's algorithm requires no additional memory. Because of its efficiency, it is one of the most popular linked list interview questions and appears frequently in algorithm textbooks.

Below is the full functional Cyclic Linked List implementation (with additional methods):

[cycle linked lists.py](#)

Summary

In this section, we explored several advanced operations that extend the functionality of linked lists beyond simple insertion and deletion. We learned how to reverse a linked list by changing the connections between nodes, concatenate multiple linked lists into a larger structure, and detect cycles that could otherwise cause infinite traversal.

We also studied two different cycle detection techniques. The first uses a set to record visited nodes, while the second uses the highly efficient **Floyd's Cycle Detection Algorithm**, which requires only constant additional memory.

These operations demonstrate the flexibility and power of linked lists as dynamic data structures. By combining fundamental operations with these advanced techniques, we can solve a wide variety of real-world problems involving dynamic collections of data.

Having completed our study of linked lists, we are now ready to explore another important linear data structure that is built upon many of the same principles: the **Stack**, a Last-In-First-Out (LIFO) structure that is widely used in function calls, expression evaluation, undo operations, and many other computing applications.

References

https://drive.google.com/file/d/1xW6GD_okYUEMMlotBxclA5RdAe2Rx8Gu/view?usp=drive_link

<https://www.geeksforgeeks.org/dsa/detect-loop-in-a-linked-list/>

Next Section:

 [9. Stacks](#)